Linux Fundamentals (Module 5)

presented by
Dr. Priyanka Chakraborty

Process

- A process is a program in execution
- A process is **born** when a program starts its executation and remains alive as long as the program is active
- After execution is complete the process is said to **die**
- When we execute the **grep** command, a process named **grep** is created
- When 2 users run the same program, there is only one program is on the disk but 2 processes are in the memory
- **Kernel is responsible for the management of processes**
- Kernel determines the time and priorities which are allocated to processes so that multiple processes can share limited CPU resources.
- Some attributes of each process are maintained by the kernel in memory in a separate structure called a process table.
- Like **inodes** for file, **process table** also stores process attributes

Process

- Two important attributes of a process are

(i) **Process Id(PID)-** Each process is uniquely identified by a unique integer called the PID which is allocated by the kernel when the process is born. We need this PID to control a process, like to kill it.

(ii) **Parent ID (PPID)-**PID of the parent process is also available as a process attribute.

- The **swapper or sched** has **PID 0** and is responsible for paging and is actually part of the kernel rather than a normal user-mode process.
- PID of **init** (stands for initialization) process is 1. It is the first process and the parent of all processes in linux.
- It is the first process which starts when a computer boots up and continues to run until system shut down.
- A shell maintains a set of environment variables like **PATH** and **SHELL**
- The shell's pathname is stored in SHELL but its PID is stored in a special variable, $$. To know the PID of your current shell
  - **$ echo $$**
  - **291**
- The PID of your log in shell can't change as long as you are logged in.

Process

- A low PID indicates that the process was initiated quite early.
- When you log out and log in again, your login shell will be assigned a different PID.
- All commands doesn't set up processes.
- Biult-in commands (Internal Commands) of the shell like pwd, cd, echo etc don't create processes.
- When you run a command like

  $ cat emp.lst

- From the keyboard, a process representing the **cat** command is started by the shell process. This cat process remains active as long as the command is active.
- The shell (which could be sh, ksh, bash or csh) is said to be the parent of cat, while cat is said to be the child of the shell.

Process

- The following command sets up 2 processes for the 2 commands.

$ Cat emp.lst | grep ‘director’

These processes like cat and grep are spawned (produced) by the shell. So, Shell is the parent process and cat, grep are children.

ps: process status

$ ps

PID	**TTY**	**TIME**	**CMD**
291	Console	0.00	bash

PID - Process Identification Number

TTY – terminal with which the process is associated (controlling terminal)

TIME – cumulative processor time that has been consumed since the process has been started

CMD – process name

Process

Full listing (-f): To get a detailed listing which also shows the parent of each process, use the –f option:

```
$ ps -f
    UID   PID  PPID  C    STIME TTY      TIME CMD
  sumit   367   291  0 12:35:16 console  0:00 vi create_user.sh
  sumit   291     1  0 10:24:58 console  0:00 -bash
  sumit   368   367  0 12:35:18 console  0:00 /usr/bin/bash -i
```

Now we can see the parent (PPID) and Owner (UID) of the process.

The login shell (PID 291) has the PPID 1, the second process of the system named as **init** process(The first process has the PID 0)

$ ps -e or –A ------- It will show all processes including user and system processes

$ ps –a ----- It will show all user processes but not the system processes

Process

- **Fields described by ps are described as:**
- **UID**: User ID that this process belongs to (the person running it)
- **PID**: Process ID
- **PPID**: Parent process ID (the ID of the process that started it)
- **C**: CPU utilization of process
- **STIME**: Process start time
- **TTY**: Terminal type associated with the process
- **TIME**: CPU time is taken by the process
- **CMD**: The command that started this process

Types of Processes

1. **Parent and Child process :** The 2nd and 3rd column of the ps –f command shows process id and parent's process id number. For each user process, there's a parent process in the system, with most of the commands having shell as their parent.

2. Orphan process : After completing its execution a child process is terminated or killed and SIGCHLD updates the parent process about the termination and thus can continue the task assigned to it.
But at times when the parent process is killed before the termination of the child process, the child processes become orphan processes, with the parent of all processes "init" process, becomes their new pid.
Q. Explain what happens to *orphan processes* in Linux.
When a process's parent exits before the child, the child becomes an **orphan**. In Linux, the init process (PID 1) adopts orphans and reaps them when they exit.

3. Zombie process: A zombie process in Unix is a process that has completed execution but still has an entry in the process table. This occurs when the parent process has not yet read the exit status of the terminated child process. The process is called a "zombie" because it is dead but still consumes system resources.
A process which is killed but still shows its entry in the process status or the process table is called a zombie process, they are dead and are not used.

How Zombie Processes are Created

When a process is created using the fork() system call, the parent process is cloned. If the parent process calls the wait() system call, it suspends its execution until the child process terminates. Upon termination, the child process sends a SIGCHLD signal to the parent, which then reads the child's exit status from the process table and removes the entry.

However, if the parent process does not call wait(), the child process's entry remains in the process table even after it has terminated, resulting in a zombie process.

4) Daemon process : They are system-related background processes that often run with the permissions of root and services requests from other processes, they most of the time run in the background and wait for processes it can work along with for ex print daemon.
When ps –ef is executed, the process with ? in the tty field are daemon processes.

Process

- **$ ps –u ----- ** The system administrator needs to use the –u option to know the activities of any user.

```
ps -u sumit
 PID TTY          TIME CMD
 378 ?            0:05 Xsun
 403 ?            0:00 Xsession
 339 pts/3        0:00 bash
 346 pts/3        0:00 vi
 347 pts/3        0:00 bash
 478 ?            0:00 dtfile
 460 pts/5        0:00 dtsessio
 467 ?            0:01 dtwm
 479 ??           0:00 dtterm
```

Daemon Process

- System processes are easily identified by the ? In the TTY column, they generally don't have a controlling terminal.
- We can't press [ctrl-c]to interrupt these processes either.
- These processes are known as **daemons** because they are called without a specific request from a user.
- Many of these **daemons** are actually sleeping and wake up only when they receive input.
- Daemons do important work for the system.
- The **lpsched** daemon controls all printing activities.
- **sendmail** handles both incoming and outgoing mails
- **inetd** daemon acts as server side of FTP and TELNET

Daemon Process

```
$ ps -e
  PID TTY      TIME CMD
    0 ?        0:01 sched
    1 ?        0:00 init
    2 ?        0:00 pageout
    3 ?        4:36 fsflush
  194 ?        0:00 syslogd
  170 ?        0:00 inetd
  231 ?        0:00 lpsched
  200 ?        0:00 cron
  247 ?        0:00 sendmail
 2931 ?        0:00 in.telne
  292 ?        0:00 dtlogin
 1436 ?        0:00 in.rlogi
 3054 pts/2    0:00 bash
 3006 ?        0:01 dtwm
 2908 pts/4    0:00 vi
 2993 pts/6    0:00 bash
```

Takes care of swapping
Parent of all shells
Part of the kernel—not exec'd
Part of the kernel—not exec'd
Logs all system messages
Server side of FTP and TELNET
The printer daemon
Schedules your jobs
Handles all your mail
Serves your TELNET requests
Serves your RLOGIN requests
Handles windows on X Window

Daemon Process

- **cron** daemon ---- running jobs periodically
- Cron is the UNIX system's chronograph, ticking away every minute.
- Unlike at and batch that are meant for one-time execution, cron executes programs at regular intervals.
- Every minute it wakes up and looks in a **crontab file** (/var/spool/cron/crontabs/kumar) for instructions to be performed at that instant.
- After executing them, it goes back to sleep, only to wake up the next minute.
- The crontab command syntax has six fields separated by space where the first five represent the time to run the task and the last one is for the command.
  - Minute (holds a value between 0-59)
  - Hour (holds value between 0-23)
  - Day of Month (holds value between 1-31)

Daemon Process

- Month of the year (holds a value between 1-12 or Jan-Dec, the first three letters of the month's name shall be used)
- Day of the week (holds a value between 0-6 or Sun-Sat, here also first three letters of the day shall be used)
- Command

➢ **<u>at AND batch: Execute Later</u>**

➢ at takes as its argument the time the job will be executed and displays the at> prompt

➢ **$ at 14:08**

at>empawk2.sh **-----**The job goes to the queue and at 2:08 p.m today, the script file empawk2.sh will be executed.

➢ **batch: Execute in batch queue**

➢ The batch command also schedules jobs for later execution but unlike at, jobs are executed as soon as the system load permits.

➢ This command does not take any arguments but uses an internal algorithm to determine the execution time.

➢ The system administrator controls the use of at and match on your system

Mechanism of Process Creation

- The entire Process creation mechanism is done through 3 system calls or functions like fork(), exec() and wait()
- **Fork()** -- A process is created with fork which creates a copy of the process that invokes it. The process image is identical to that of the calling process except its PID. Child gets a new PID.
- **Exec()** – The forked child needs to overwrite its own image with the code and data of the new program. This mechanism is called as exec, and the child process is said to exec a new program. No new process is created here, the PID and PPID of the exec'd process remain unchanged.
- **Wait()** – The parent then executes the wait system call to wait for the child process to complete. It picks up the exit status of the child and then continues its other functions.

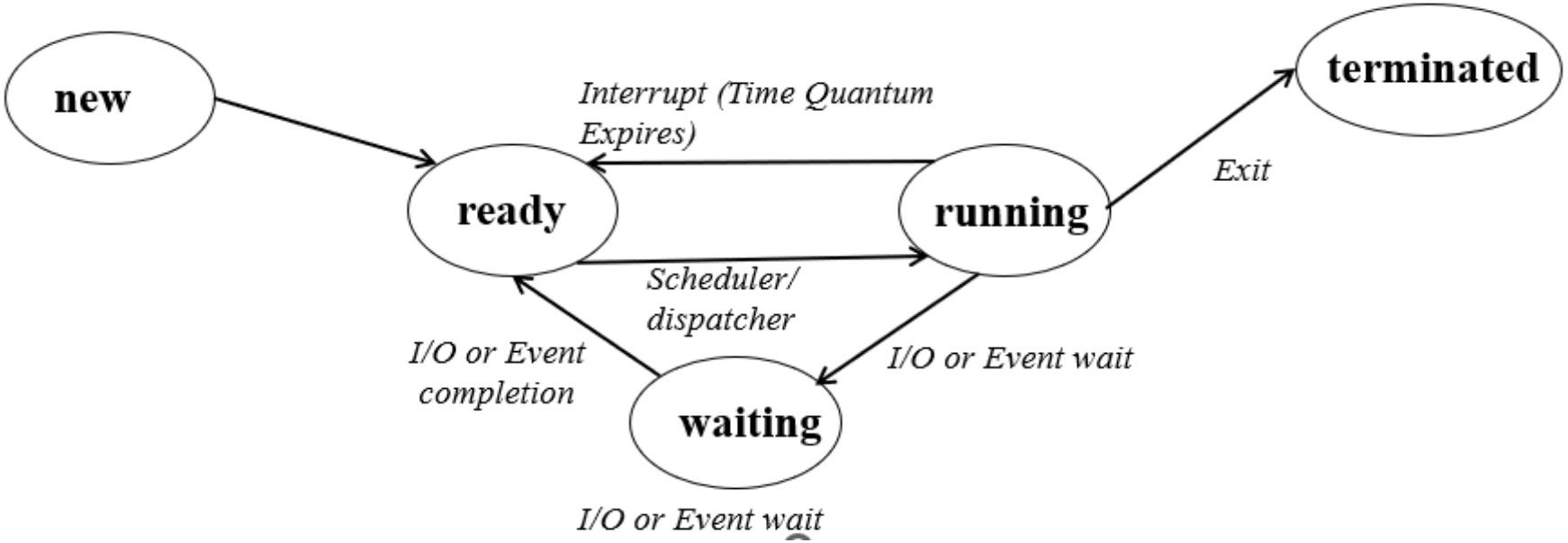
How the Shell is created

- When the system moves to the multiuser mode, init forks and execs a getty for every active communication port.
- Each one of these gettys prints the login prompt on the respective terminal and then goes off to sleep.
- When a user attempts to log in, getty wakes up and fork-execs the login program to verify the log in name and password entered.
- On successful log in, the login forks-execs the process representing the login shell.
- **init** $\xrightarrow{\text{fork-exec}}$ **getty** $\xrightarrow{\text{fork-exec}}$ **login** $\xrightarrow{\text{fork-exec}}$ **shell**
- init goes off to sleep, waiting for the death of its children.
- When the user logs out, her shell is killed and the death is intimated to init.
- Init then wakes up and spawns another getty .

Process States and Zombies

<u>Process State</u> →*(i) New*- The process is being created. *(ii) Running* – Instructions are being executed. *(iii) Waiting* – The process is waiting for some event to occur such as an I/O completion or reception of a signal. (iv) Ready – The process is waiting to be assigned to a processor. (v) *Terminated* - The process has finished execution.

<u>Fig. Process States</u>



Process States and Zombies

- While the process is running, it may invoke a disk I/O operation when it has nothing to do except wait for the I/O to complete.
- The process then moves to the sleeping/ waiting state to be woken up when the i/o operation is over.
- A process can also be suspended by pressing a key (usually, [Ctrl-z]).
- Processes whose parents don't wait for their death move to the **zombie** state.
- When a process dies, it immediately moves to the zombie state. It remains in this state until the parent picks up the child's exit status (the reason for waiting) from the process table.
- A zombie is a harmless dead child but we can't kill it.
- It's also possible for the parent itself to die before the child dies.
- Then the child becomes an **orphan** and the kernel makes **init** the parent of all orphans.
- When this adopted child dies, **init** waits for its death.

Difference between Zombie, Orphan, and Daemon Processes

Zombie Process	Orphan Process	Daemon Process
A Zombie is a process that has completed its task but still, it shows an entry in a process table.	A child process that remains running even after its parent process is terminated or completed without waiting for the child process execution is called an orphan.	A daemon process is a system-related process always running in the background.
Zombie process states always indicated by Z	The orphan process was created unknowingly due to a system crash.	Daemon process state indicated by ? in the field of *tty* column in the output
A zombie process has controlling terminals	An Orphan process has controlling terminals.	A daemon process does not have controlling terminals.
The zombie process treated as dead they are not used for system processing	An orphan process is a computer process even after their parent terminates init is become a parent and continue the remaining task.	A program that runs for a long time makes them as a daemon process and runs it in the background.
To remove the zombie process execute the kill command.	Terminate the Orphan process use the SIGHUP signal.	Daemon process only when system shutdown.

Running Jobs in Background

- In a multi-tasking system, a user may run multiple processes at a time.
- Since, there can be only one job in the foreground, the rest of the jobs have to run in the background.
- Two approaches are there- (i) with the shell's **& operator** and the **nohup** command.
- **nohup** permits a user to log out while his jobs are running, but **&** doesn't allow that (except in the C shell and Bash).
- **(i) & No Logging Out**

The & is the shell's operator used to run a process in the background. The parent doesn't wait for the child's death. Just terminate the command line with an &, the command will run in the background:

$ sort –o emp.lst emp.lst &

550 ----- The job's PID

The shell immediately returns a number- the PID of the invoked command(550)

Running Jobs in Background

- Using an & a user can run as many jobs in the background as the system load permits.
- Hence, a user can run the time-consuming and low-priority jobs in the background, and run the important jobs in the foreground.

(ii) nohup: log out safely

Background jobs cease to run, when a user logs out (the C shell and Bash excepted). It happens because his shell is killed. When the parent is killed, its children are also normally killed.

The nohup (no hangup) command, when prefixed to a command, permits execution of the process even after the user has logged out. We must use the & with as well.

$ nohup sort emp.lst &

586

Sending output to nohup.out

When the nohup command is run in these shells, nohup sends the standard output of the command to the file nohup.out.

Running Jobs in Background

- When we use ps command after using nohup from another terminal the output will be like
- **$ ps –f –u kumar**
- UID PID PPID C STIME TTY TIME COMMAND
- kumar 586 1 45 14:52:09 01 0:13 sort emp.lst
- The shell died (rather was killed) on logging out but its child (sort) didn't; it turned into an orphan.
- The kernel handles such situations by reassigning the PPID of the orphan (sort) to the system's init process (PID 1)- the parent of all shells.
- When the user logs out, init takes over the parentage of any process run with nohup.
- Thus, we can kill a parent (the shell) without killing its child(sort).
- If we run more than one command in a pipeline, you should use the nohup command at the beginning of each command in the pipeline:
- **$ nohup grep 'director' emp.lst & | nohup sort &**

Job Execution with low priority

- **nice Command–** This command is used to start a new process with a specific priority, known as the "nice value." A higher nice value lowers the process's priority, while a lower (negative) nice value increases it. Processes with higher priority receive more CPU time.

To set the priority of a process

- To start a new process with a specific nice value, use the nice command:

  nice -10 gnome-terminal

- A higher nice value implies a lower priority.

To set the negative priority for a process

- To give a process higher priority by assigning a negative nice value, use:
- nice --10 gnome-terminal

renice command

Changing priority of the running process.

- To modify the priority of an already running process, use the ‘renice’ command with the process ID (PID):
- sudo renice -n 15 -p 77982

To change the priority of all programs of a specific group.

- You can also adjust the priority of all processes within a specific group by specifying the group ID (GID):
- renice -n 10 -g 4

To change the priority of all programs of a specific user.

To change the priority of all processes belonging to a specific user, use the renice command with the user ID (UID):

sudo renice -n 10 -u 2

- **kill: Premature Termination of a process—**
- kill 121 122 125 132 138
- If all the processes have the same parent, we may simply kill the parent in order to kill all its children.
- When we use nohup with a set of commands and log out, we can’t kill the parent as **init** is the parent and we can’t kill **init**.